

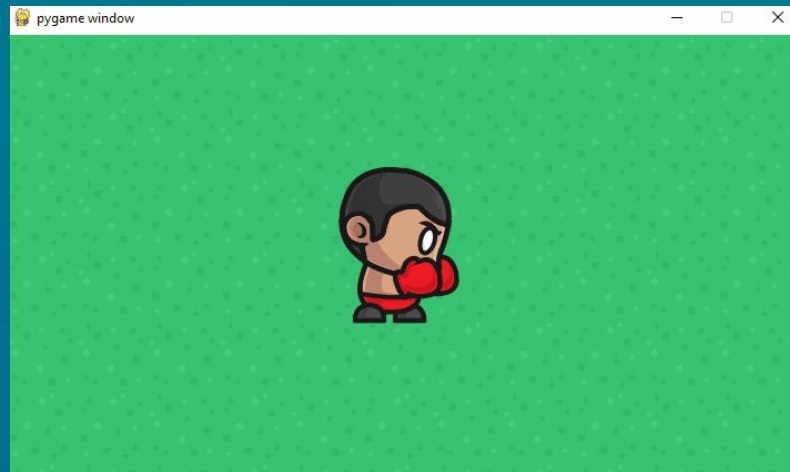


Biblioteca Pygame

Introducción

Pygame es un contenedor de Python para la biblioteca **SDL**, que significa Simple DirectMedia Layer.

SDL brinda acceso multiplataforma a los componentes de hardware multimedia subyacentes de su sistema, como sonido, video, mouse, teclado y joystick.



Características

Esta librería te permite:

- Levantar imágenes en formato PNG, BMP, JPG, entre otros.
- Usar sistemas de sonido con audio en formato WAV, OGG, MP3.
- Realizar operaciones relacionadas con el gestor de ventanas.
- Manejar eventos de la aplicación.
- Manejo de dispositivos de entrada tales como mouse y teclado.
- Crear temporizadores para realizar acciones.
- Manejar colisiones entre objetos.
- Manejar un sistema de Sprites para elementos del juego.



Ciclo de vida de un juego



Pensá en un juego como una película que se reproduce constantemente. Cada fotograma es una actualización del estado del juego. En Pygame, esto se logra con un bucle infinito que constantemente revisa si hay eventos (como pulsar una tecla), actualiza la posición de los objetos en pantalla y luego vuelve a dibujar todo.

Estructura básica de un juego en Pygame

1. **Inicialización:** Importamos Pygame, inicializamos todo y creamos la ventana principal.
2. **Bucle principal:** Este bucle se ejecuta mientras el juego esté activo.
3. **Gestionar eventos:** Revisamos si se han producido eventos, como pulsar una tecla o cerrar la ventana.
4. **Actualizar el juego:** Cambiamos las posiciones de los objetos, calculamos físicas, etc.
5. **Dibujar en pantalla:** Dibujamos todos los elementos en la pantalla.
6. **Actualizar la pantalla:** Mostramos lo que hemos dibujado.

Ejemplo de código

En este ejemplo se importa la librería para luego inicializarla. Como siguiente paso se crea un objeto que se usará para la pantalla o display y luego se gestiona el bucle principal del juego en el cual se evalúan cada uno de los eventos para realizar una acción específica.

```
import pygame

# Inicializamos Pygame
pygame.init()

# Creamos la ventana principal
pantalla = pygame.display.set_mode((800, 600))

# Bucle principal del juego
while True:
    # Gestionar eventos (por ejemplo, cerrar la ventana)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            pygame.quit()
            quit()

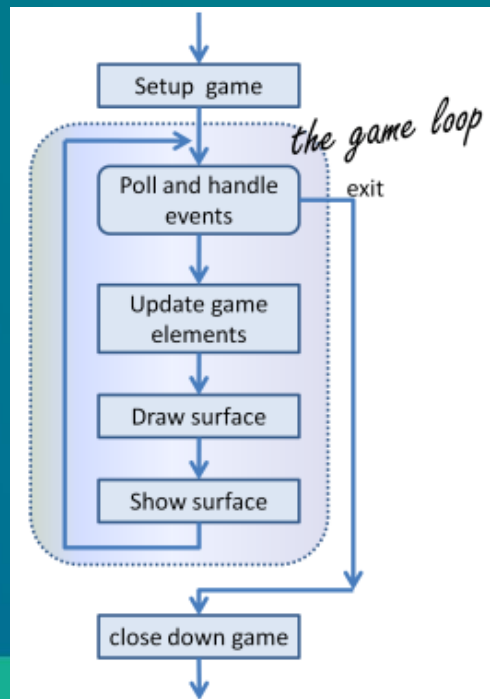
    # Actualizar el juego (por ahora, nada)

    # Dibujar en pantalla (por ahora, rellenamos de un color)
    pantalla.fill((0, 255, 0)) # Verde

    # Actualizar la pantalla
    pygame.display.update()
```

Ciclo de vida del juego

Luego de que se configura el juego, Se manejan eventos y se actualizan los elementos del juego (su posición, etc). Luego de eso, se dibujan las superficies en pantalla y se muestran hasta que se ejecute el evento para cerrar el juego.



La Ventana Principal: Nuestro Lienzo

La ventana principal es el lienzo donde vamos a pintar nuestro juego. Es como un marco en el que encajamos todas las imágenes y elementos.

```
pantalla = pygame.display.set_mode((800, 600))
```

El método `set_mode()` recibe por parámetro una tupla con la dimensión en píxeles de la pantalla de nuestro juego, por ejemplo, si queremos que nuestra ventana sea de 800 de ancho X 600 de alto px haremos:

```
dimension_pantalla = (800, 600)  
set_mode(dimension_pantalla)
```


Configuraciones Adicionales

A nuestro display le podemos configurar unos datos extra para mejorar el aspecto de nuestra ventana del juego, algunos de ellos son:

Título: `pygame.display.set_caption("Mi Juego")`

Icono: `pygame.display.set_icon(ícono)` (donde **ícono** es una superficie de Pygame)

Modo a pantalla completa: `pygame.display.set_mode((800, 600), pygame.FULLSCREEN)`

Ventana redimensionable: `pygame.display.set_mode((800, 600), pygame.RESIZABLE)`

Imágenes

El módulo de imagen contiene funciones para cargar y guardar imágenes, así como para **transferir superficies** a formatos que puedan utilizar otros paquetes.

Tenga en cuenta que no existe una clase Image; **una imagen se carga como un objeto Surface**. La clase Surface permite la manipulación (dibujar líneas, configurar píxeles, capturar regiones, etc.).

Para cargar una superficie, utilizamos lo siguiente:

- [pygame.image.load](#)
- `imagen = pygame.image.load("01.png")` Para cargar superficies.
- `imagen = pygame.transform.scale( imagen,(ancho,alto))` Para escalar las superficies.

Imágenes

Algunos Módulos:

- `pygame.image.load`: Nos permite cargar imágenes.
- `pygame.image.save`: Guarda una imagen a archivo
- `pygame.image.get_sdl_image_version`: Obtener el número de versión de la biblioteca `SDL_Image` que se está utilizando.
- `pygame.image.get_extended`: Prueba si se pueden cargar formatos de imagen extendidos.
- `pygame.transform.scale( imagen,(ancho,alto))` Nos permite escalar superficies.

Imágenes

Algunos formatos que podés cargar:

- BMP
- GIF (non-animated)
- JPEG
- LBM (and PBM, PGM, PPM)
- PCX
- PNG
- PNM
- SVG (limited support, using Nano SVG)
- TGA (uncompressed)
- TIFF
- WEBP
- XPM

Imágenes

Solo se puede guardar en
menos formatos:

- BMP
- JPEG
- PNG
- TGA

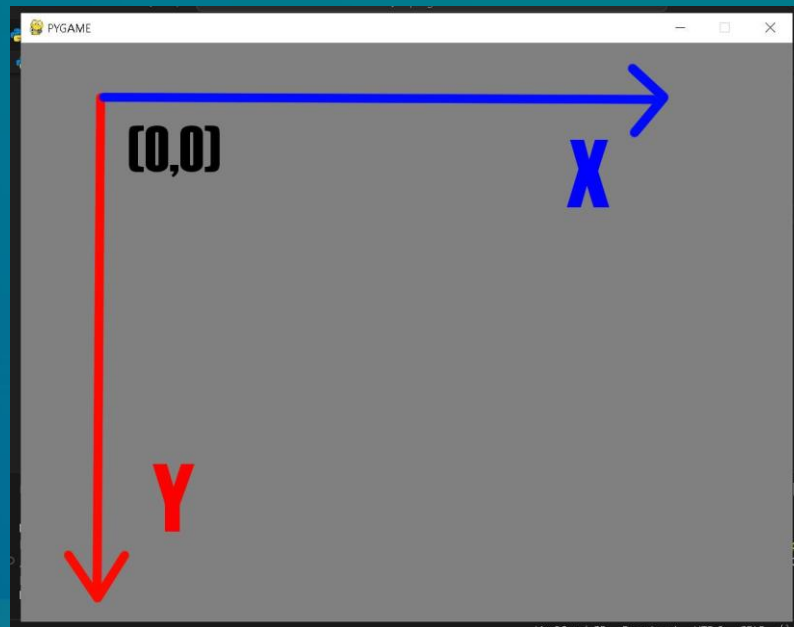


Posicionamiento

-La pantalla de un juego en pygame se representa como una cuadrícula donde cada punto tiene un valor, como un plano cartesiano.

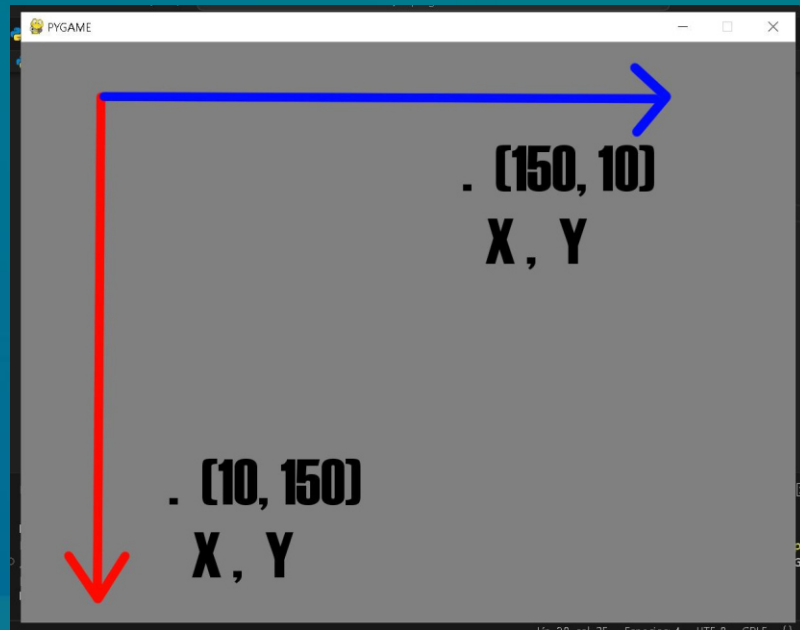
-La esquina superior izquierda de la pantalla tiene valor (0,0)

- . X: representa la posición horizontal
- . Y: representa la posición vertical



Posicionamiento

- A medida que aumentas el valor de X, te desplazas hacia la derecha
- A medida que aumentas el valor de Y, te desplazas hacia abajo
- Cada punto en pantalla es una intersección entre valores de los ejes (X,Y)



Figuras Geométricas

Sobre una superficie se pueden dibujar diferentes formas (círculos, rectángulos, elipses, líneas). Estas figuras son superficies en sí y para crearlas tendremos que tener en cuenta ciertos argumentos:

Para crear un Rectángulo:

```
color_verde = (0, 255, 0)
coordenadas = (30, 30, 60, 60)
rectangulo = pg.draw.rect(screen, color_verde, coordenadas)
```


Figuras Geométricas

el método **.rect** de pygame nos permite setear una superficie de rectángulo. el primer argumento será la superficie de display de nuestro juego, ya que en ese mismo lugar lo dibujaremos, como segundo argumento pondremos el color de nuestro rectángulo, el color tiene que estar expresado como una tupla con formato numérico tal como si se tratase de un RGB (de hecho los números representan eso mismo) siendo, por ejemplo:

ROJO = (255, 0, 0) | VERDE = (0, 255, 0) | AZUL = (0, 0, 255)

Como siguiente argumento pondremos las coordenadas donde queremos que se dibuje dicho rectángulo, debe estar expresado como una tupla de 4 elementos que representan las coordenadas, ejemplo:

coordenadas = (30, 30, 100, 60) siendo:

coordenadas = (EJE_X, EJE_Y, ANCHO_RECTANGULO, ALTO_RECTANGULO)

El alto y ancho está expresado en píxeles.

Figuras Geométricas

En el caso del círculo, es similar a lo que hicimos con el rectángulo.
Sus argumentos serán:

La pantalla donde lo dibujaremos

El color de la figura (en RGB)

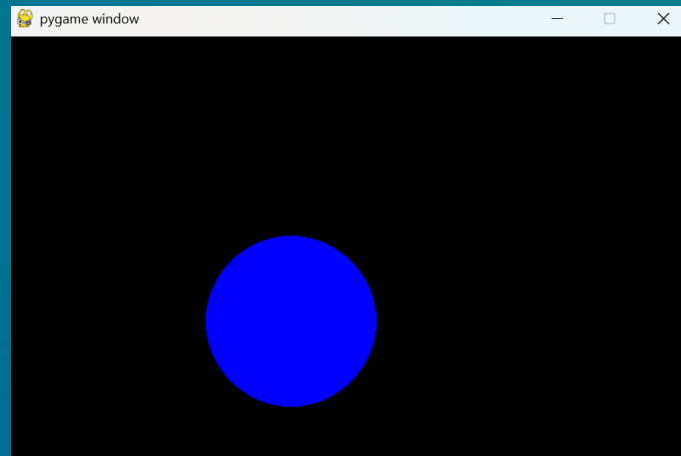
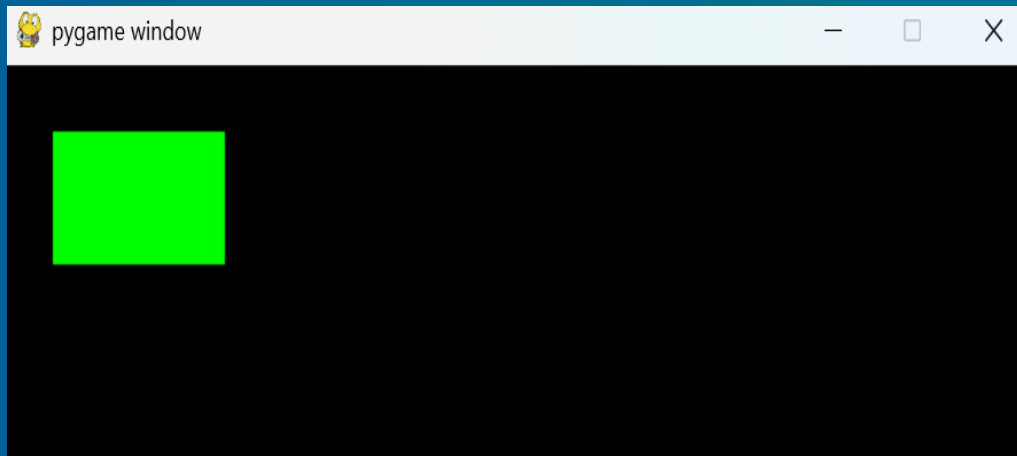
Las coordenadas del centro del círculo

El radio del círculo

```
color_azul = (0, 0, 255)
coordenada_centro = (250, 250)
radio = 75
circulo = pg.draw.circle(screen, color_azul, coordenada_centro, radio)
```

Figuras Geométricas

y así se verán las dos figuras geométricas creadas



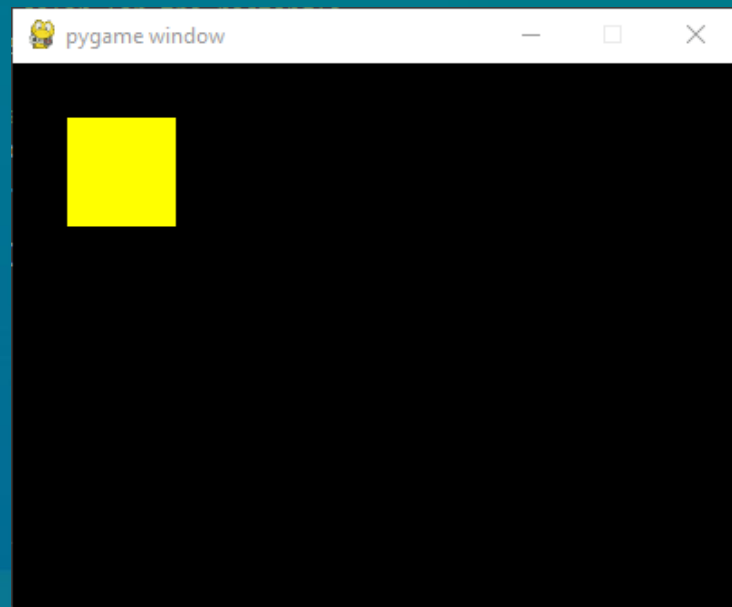


Superficies y sonidos

Superficies y Rectángulos

Al utilizar **Pygame**, generalmente se utilizan superficies para representar la apariencia del objeto y su posición en la pantalla. Todos los objetos, textos e imágenes que creamos en Pygame se crean utilizando superficies.

Crear superficies en pygame es bastante fácil. Solo tenemos que pasar la altura y el ancho con una tupla al método `pygame.surface()`.



Superficies y Rectángulos

Podemos utilizar varios métodos para formatear nuestra superficie como queramos. Por ejemplo, podemos utilizar `pygame.draw()` para dibujar formas, podemos utilizar el método `surface.fill()` para rellenar la superficie. Ahora, la implementación de estas funciones. Analicemos la sintaxis y los parámetros.

```
# Importing the library
import pygame
import time

# Initializing Pygame
pygame.init()

# Creating the surface
sample_surface = pygame.display.set_mode((400,300))

# Choosing red color for the rectangle
color = (255,255,0)

# Drawing Rectangle
pygame.draw.rect(sample_surface, color,
                 pygame.Rect(30, 30, 60, 60))

# The pygame.display.flip() method is used
# to update content on the display screen
pygame.display.flip()
```

Superficies y Rectángulos

- **surface** (**Surface**) -- superficie sobre la que dibujar.
- **color** (**Color** o *int* o *tupla(int, int, int, [int])*) -- color para dibujar, el valor alfa es opcional si se usa una tupla (RGB[A]).
- **rect** (**Rect**) -- Rectángulo para dibujar, posición y dimensiones.
- **width** (*int*) – (opcional) se utiliza para el grosor de la línea o para indicar que se debe rellenar el rectángulo (no debe confundirse con el valor de ancho del parámetro rect).
 - if `width == 0`, (default) fill the rectangle
 - if `width > 0`, used for line thickness
 - if `width < 0`, nothing will be drawn

Superficies y Rectángulos

Veamos cómo dibujar un rectángulo con `pygame.draw.rect`.

```
rect(surface, color, rect) # Rect
rect(surface, color, rect, width=0, border_radius=0, border_top_left_radius=-1,
border_top_right_radius=-1, border_bottom_left_radius=-1, border_bottom_right_radius=-1) # Rect
```

```
# Importing the library
import pygame
import time

# Initializing Pygame
pygame.init()

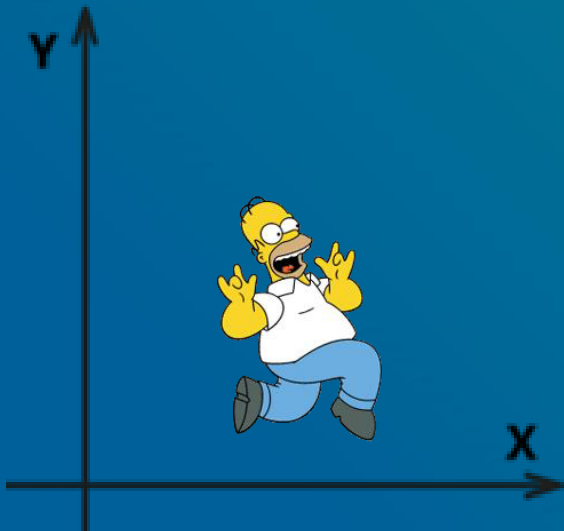
# Creating the surface
sample_surface = pygame.display.set_mode((400,300))

# Choosing red color for the rectangle
color = (255,255,0)

# Drawing Rectangle
pygame.draw.rect(sample_surface, color,
                 pygame.Rect(50, 30, 60, 60))

# The pygame.display.flip() method is used
# to update content on the display screen
while True:
    pygame.display.flip()
```


Movimiento



- El movimiento de nuestros objetos en pygame se logra actualizando las coordenadas de los mismos en los ejes X e Y dentro del bucle del juego
- Aumentar X mueve el objeto a la derecha, disminuirlo, hacia la izquierda
- Aumentar Y mueve el objeto hacia abajo, disminuirlo, hacia arriba

Movimiento

```
imagen_vertical = pygame.Surface((100,100)) #creo una superficie
imagen_vertical.fill(VERDE)
rectangulo_vertical = imagen_vertical.get_rect() # -> obtiene el rectangulo de la imagen

while True:
    clock.tick(FPS)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    PANTALLA.fill(NEGRO)
    PANTALLA.blit(imagen_vertical) # actualizo superficie en pantalla

    rectangulo_vertical.y += 10 # Aumento su posicion en el eje Y; avanza hacia abajo
    if rectangulo_vertical.top > ALTO:
        rectangulo_vertical.bottom = 0 # cuando llega al limite de la pantalla, lo mando arriba de nuevo

    pygame.display.flip()
```

Sonidos | Música de fondo

Para poder usar música en nuestro juego, necesitaremos importar el módulo **mixer** de la biblioteca Pygame

Una vez importado tenemos que inicializarlo llamando al método `.init()` para poder utilizarlo

```
import pygame.mixer as mixer  
mixer.init()
```



Sonidos | Música de fondo

Una vez inicializado, en caso de querer cargar una música de fondo para nuestro juego, podemos usar en conjunto 3 métodos del módulo music perteneciente a mixer. Como argumento le pondremos la ruta al archivo de audio para cargarlo.

```
mixer.music.load('./assets/snd/select.mp3')
```

Sonidos | Música de fondo

Como siguiente paso configuraremos su volumen con valores entre 0 y 1, valiéndonos números flotantes, por ejemplo volumen al 40%

```
mixer.music.set_volume(0.4)
```

Este método nos permitirá modificar el volumen según lo necesitemos en nuestro juego, ya sea con algún control de volumen o simplemente aplicar un volumen por defecto que no varíe.

Sonidos | Música de fondo

Finalmente para reproducir el sonido usaremos el método `play()` invocándolo donde lo necesitemos

```
mixer.music.play()
```

Esto está diseñado para reproducir **archivos de música más largos** como canciones completas. Nos ofrece funciones específicas para controlar la reproducción de la música, tal como los métodos:

`mixer.music.pause()` | `mixer.music.unpause()` | `mixer.music.stop()`

La limitación de esto es que puede manejar un archivo a la vez, si se intenta cargar un nuevo archivo, el anterior se detendrá.

Sonidos | Efectos de sonido

Otra forma de hacerlo, por ejemplo si queremos poner sonidos a los personajes de nuestro juego, es mediante la clase Sound, cuyo primer argumento será la ruta a nuestro audio.

```
sonido = mixer.Sound(ruta_al_audio)
sonido.set_volume(0.4)
sonido.play()
```

Esto es ideal para efectos de sonido cortos, efectos de juego (disparos, explosiones, etc). Se pueden crear múltiples instancias de Sound y reproducir varios sonidos simultáneamente como también controlar el volumen de todas también de manera simultánea. Es una forma más flexible para controlar múltiples sonidos al mismo tiempo.



Eventos y Colisiones

Eventos

-Los **eventos** en Pygame son interacciones que se generan en respuesta a acciones del usuario. Podemos capturarlos y manejarlos. Hacen el juego interactivo

-Capturar eventos

```
while True:
    for event in pygame.event.get():
```

Usamos `pygame.event.get()` para obtener la lista de eventos que han ocurrido y procesarlos. Dentro del WHILE con un for each podemos recorrer los eventos 1 por 1

Eventos

```
while True:
    #Obtengo la lista de eventos que estan ocurriendo y la recorro
    for event in pygame.event.get():
        #Verifico el tipo del evento para saber si coincide con lo que busco
        if event.type == QUIT:
            #Ejecuto las acciones correspondientes con el evento
            pygame.quit()
            sys.exit()
```

```
sys.exit()
```

-Haciendo .type a cada evento discernimos cual es.

-El evento del tipo QUIT sirve para cerrar el juego.

Eventos (del teclado)

-Los **eventos del teclado** permiten detectar cuando el usuario presiona o suelta una tecla. Pygame tiene dos eventos principales para el teclado:

- . `pygame.KEYDOWN`: Se activa cuando el usuario presiona una tecla.
- . `pygame.KEYUP`: Se activa cuando el usuario suelta una tecla.

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_LEFT: # Detecta si se presiona la flecha izquierda
        print("Flecha izquierda presionada")
    if event.key == pygame.K_SPACE: # Detecta si se presiona la barra espaciadora
        print("Espacio presionado")
```

Eventos (del teclado)

También puedes detectar múltiples teclas a la vez usando `pygame.key.get_pressed()`, que retorna una lista de todas las teclas presionadas.

```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    mover_izquierda()
if keys[pygame.K_RIGHT]:
    mover_derecha()
```

Eventos (del mouse)

-Al igual que el teclado, Pygame permite capturar eventos del mouse, por ejemplo, si el click fue presionado o soltado

pygame.MOUSEBUTTONDOWN: Cuando se presiona un botón del mouse.

```
if event.type == pygame.MOUSEBUTTONDOWN:
    posicion = (event.pos) #reasigno la tupla posicion al click
    if event.type == pygame.MOUSEBUTTONDOWN:
        if event.button == 1: # Botón izquierdo
            print("Clic izquierdo detectado")
        elif event.button == 3: # Botón derecho
            print("Clic derecho detectado")
```

event.pos -> me devuelve una tupla con los valores X, Y donde se hizo click

event.button -> nos devuelve un numérico que representa el botón del mouse donde se hizo click

Eventos Propios

Podemos crear nuestros propios eventos, por ejemplo eventos de tiempo que nos pueden servir para determinadas acciones tales como ejecutar una acción solo cuando se haya cumplido determinado tiempo o controlar los FPS.

Eventos Propios

para ello tenemos que definirlo usando `pygame.USEREVENT`.

Como el `USEREVENT` representa un número entero definido, podemos sumarle un valor entero y tener nuestro propio evento. Como haremos un evento de tiempo, podemos crear una variable con el valor del tiempo en milisegundos para que cada un segundo se ejecute.

```
import pygame as pg

evento_ticks_1 = pg.USEREVENT + 1
tiempo_milisegundos = 1000
```

Eventos Propios

Luego lo que nos queda es asignar nuestro evento al timer de pygame con el método `set_timer()`

```
pg.time.set_timer(evento_ticks_1, tiempo_milisegundos)
```

Esto nos permitirá iterar luego los eventos y verificar si hay algún evento del mismo tipo que el evento que creamos nosotros para poder realizar alguna acción específica como imprimir algún mensaje

```
while True:
    for evento in pg.event.get():
        if evento.type == evento_ticks_1:
            print('Ya paso un segundo...')
```


Colisiones

La funciones de *collide* se gestionan desde el módulo `pygame.Rect`, que es un objeto de la librería Pygame y sirve para almacenar coordenadas rectangulares en un programa.

La funciones *collide* se utilizan para observar y reaccionar a las colisiones que se hacen entre dos rectángulos de un programa. Estas 2 funciones *collide* en Pygame implican colisionar al menos dentro de un rectángulo.



Colisiones

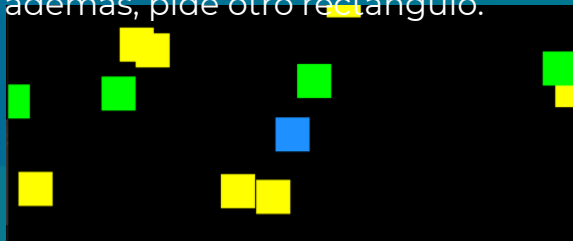
Función `pygame.Rect.collidepoint`

La función `pygame.Rect.collidepoint` permite conocer si un punto ha colisionado dentro de un rectángulo. La función `collidepoint(x,y)` resulta en un booleano cuando se aplica un rectángulo y, por tanto, pide los puntos para comprobar si colisiona

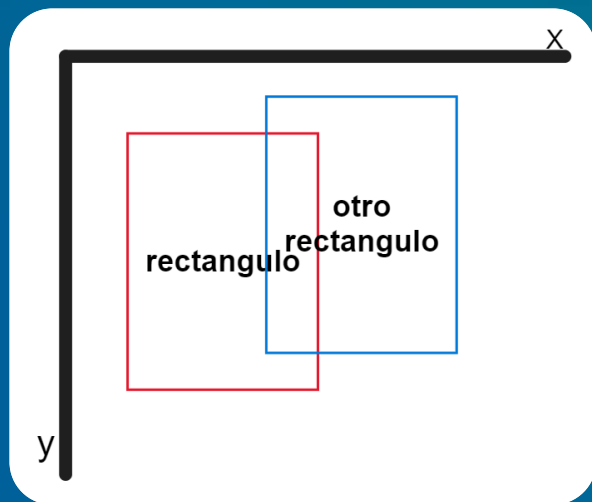


Función `pygame.Rect.colliderect`

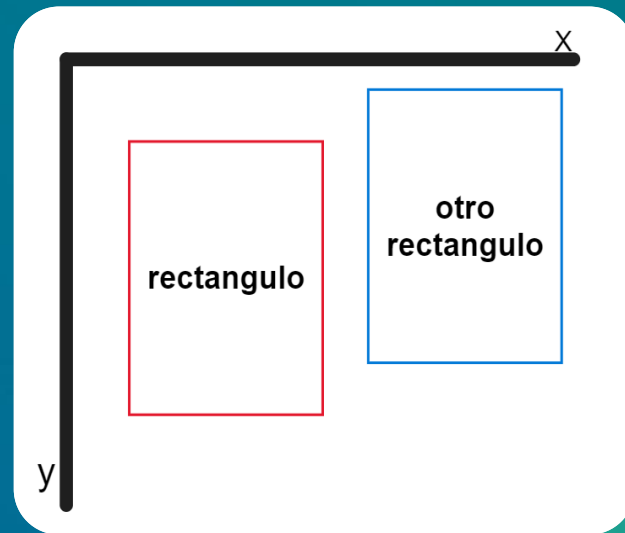
La función del módulo de Rect de Pygame, `pygame.Rect.colliderect`, comprueba si dos rectángulos se superponen, lo que quiere decir que, si esto llega a pasar, es que han colisionado uno con el otro. Si colisionan, puede ejecutarse un rebote entre ellos. La función `colliderect(Rect)` resulta en un booleano cuando se aplica a un rectángulo y, además, pide otro rectángulo.



```
# Retorna True  
rectangulo.colliderect(otro_rectangulo)
```



```
# Retorna False  
rectangulo.colliderect(otro_rectangulo)
```



Ejemplo de Colisión y cambio de color

```
while True:
    clock.tick(FPS)
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    PANTALLA.fill(NEGRO)
    PANTALLA.blit(imagen_vertical, rectangulo_vertical) # -> superpone arriba de pantalla
    PANTALLA.blit(imagen_horizontal, rectangulo_horizontal)

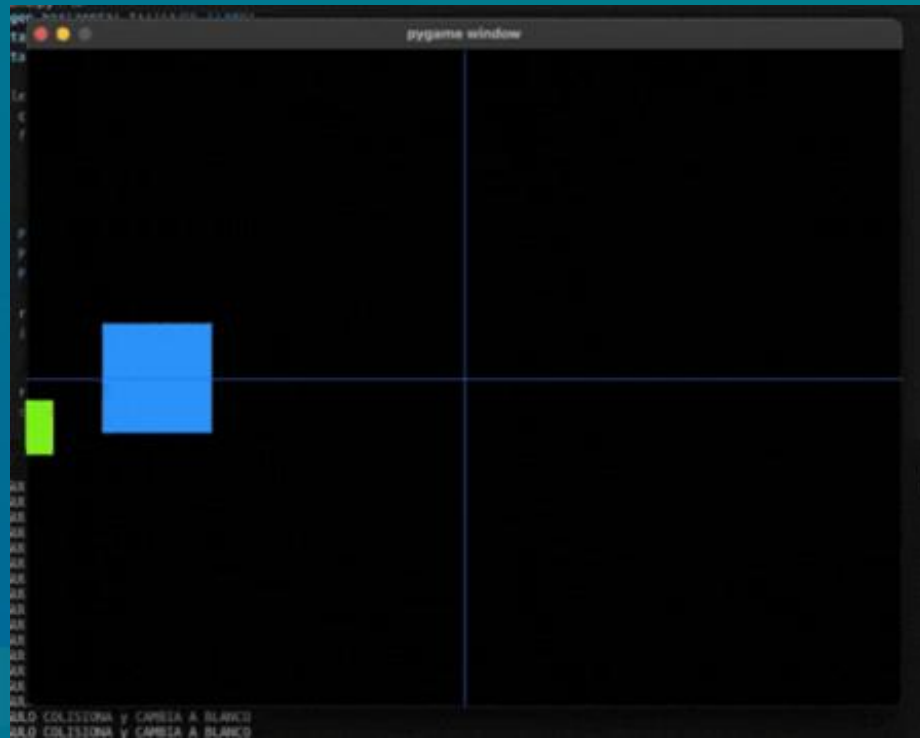
    rectangulo_vertical.y += 10
    if rectangulo_vertical.top > ALTO:
        rectangulo_vertical.bottom = 0

    rectangulo_horizontal.x += 10
    if rectangulo_horizontal.left > LARGO:
        rectangulo_horizontal.right = 0

    if rectangulo_horizontal.colliderect(rectangulo_vertical):
        print("El RECTANGULO COLISIONA y CAMBIA A BLANCO")
        imagen_horizontal.fill(BLANCO)

    else:
        imagen_horizontal.fill(AZUL_CLARO)

    pygame.draw.line(PANTALLA, AZUL, (400,0),(400, 800),1)
    pygame.draw.line(PANTALLA, AZUL, (0,300),(800, 300),1)
    pygame.display.flip()
```





Diccionarios

Representación de objetos mediante diccionarios

Podemos crear objetos para nuestro juego tales como nuestro personaje jugable, ítems, enemigos, trampas, etc.

Para ello, con lo que sabemos de Pygame nos crearemos un diccionario de nuestro personaje jugable donde especificaremos la imagen que usaremos como sprite, su rectángulo, posición y alguna otra característica que necesitemos

Crear personaje con diccionario

Por ejemplo, creamos un diccionario vacío para nuestro personaje y asignamos claves para los elementos que necesitamos tales como su superficie (la imagen cargada que usaremos), su rectángulo que usaremos para posibles colisiones el cual sacamos sus datos de su superficie, y la posición del rectángulo, opcionalmente podemos definir una clave de puntaje si así lo requerimos.

```
dict_personaje = {}  
dict_personaje["surface"] = pygame.image.load("imagen_homero.png")  
dict_personaje["surface"] = pygame.transform.scale( dict_personaje["surface"], (ancho, alto))  
dict_personaje["rect_pos"] = pygame.Rect(x, y, 200, 200)  
dict_personaje["rect"] = pygame.Rect((x+ancho/2)-10, y+90, 40, 20)  
dict_personaje["score"] = 0
```

Actualizar rectángulo y posición

Podemos actualizar la posición de la superficie del rectángulo de nuestro personaje para generar el efecto de movimiento en la dirección que lo necesitemos sumando enteros en el atributo `.x` o `.y` de la siguiente manera:

```
dict_personaje["rect_pos"].x = dict_personaje["rect_pos"].x + 30 # pixeles  
dict_personaje["rect"].x = dict_personaje["rect"].x + 30 # pixeles
```

En este caso, a nuestro personaje le aumentamos su coordenada X del rectángulo en 30 píxeles para que en algún momento de nuestro juego podamos dibujarlo 30 píxeles corrido hacia la izquierda y generar movimiento

Dibujar personaje

Para dibujarlo en nuestra ventana principal del juego, usaremos el método `blit()` de nuestro objeto ventana al cual le pasaremos como argumento la superficie de nuestro personaje y la posición del rectángulo de nuestro personaje.

```
ventana_ppal.blit(dict_personaje["surface"], dict_personaje["rect_pos"])
```

Conjuntamente en nuestro bucle principal podemos llamar el código de la siguiente manera:

Inicializar ventana principal y nuestro personaje

```
main_juego.py > ...
1 import pygame
2 import colores
3
4 ANCHO_VENTANA = 800
5 ALTO_VENTANA = 500
6
7 pygame.init()
8 ventana_ppal = pygame.display.set_mode((ANCHO_VENTANA, ALTO_VENTANA))
9 pygame.display.set_caption("PYGAME HOMERO COME DONAS")
10
11 # TIMER
12 timer = pygame.USEREVENT + 0
13 pygame.time.set_timer(timer, 100)
14
15 # player = personaje.crear(ANCHO_VENTANA/2, ALTO_VENTANA-200, 200, 200)
16
17 dict_personaje = {}
18 dict_personaje["surface"] = pygame.image.load("imagen_homero.png")
19 dict_personaje["surface"] = pygame.transform.scale(dict_personaje["surface"], (200, 200))
20 dict_personaje["rect_pos"] = pygame.Rect(ANCHO_VENTANA/2, ALTO_VENTANA-200, 200, 200)
21 dict_personaje["rect"] = pygame.Rect(
22     (ANCHO_VENTANA/2 + 200/2)-10, ALTO_VENTANA-200+90, 40, 20)
23
```

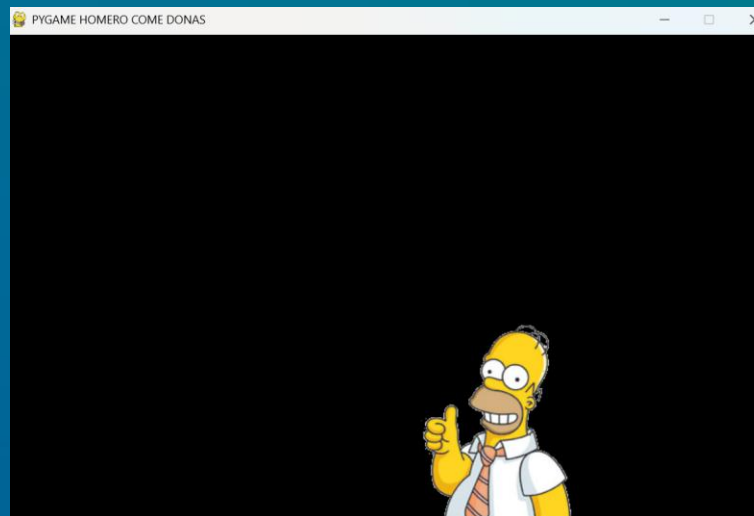
Dibujar nuestro personaje en la ventana

```
flag_run = True
while flag_run:
    lista_eventos = pygame.event.get()
    for evento in lista_eventos:
        if evento.type == pygame.QUIT:
            flag_run = False

    ventana_ppal.fill(colores.NEGRO)
    # Actualizamos la pantalla del personaje
    ventana_ppal.blit(dict_personaje["surface"], dict_personaje["rect_pos"])

    pygame.display.flip()
pygame.quit()
```

Resultado final



Mejoras en nuestro código

Todo esto se puede mejorar aplicando funciones que nos permitan crear el diccionario de nuestro personaje, actualizarlo en la ventana y dibujarlo, para evitar tener que repetir código y facilitarnos la tarea y prolijidad en nuestro trabajo.

Como primer paso lo ideal es crear un módulo para las funciones de nuestro personaje, en este caso lo llamaremos `homero.py`

`homero.py`

Crear el diccionario de nuestro personaje

Respecto a crear el diccionario de nuestro personaje, podemos tener una función llamada `crear()` la cual reciba como parámetro las coordenadas X e Y, el ancho de la imagen redimensionada y el alto.

```
def crear(x,y,ancho,alto):  
    dict_personaje = {}  
    dict_personaje["surface"] = pygame.image.load("imagen_homero.png")  
    dict_personaje["surface"] = pygame.transform.scale( dict_personaje["surface"],(ancho,alto))  
    dict_personaje["rect_pos"] = pygame.Rect(x,y,200,200)  
    dict_personaje["rect"] = pygame.Rect((x+ancho/2)-10,y+90,40,20)  
    dict_personaje["score"] = 0  
  
    return dict_personaje
```

Actualizar personaje

También podemos crear una función llamada `update()` que se encargue de actualizar la posición de nuestro personaje, la cual reciba como parámetro el diccionario del personaje y cuando va a incrementar o decrementar en X

```
def update(personaje, incremento_x):  
    nueva_x = personaje["rect_pos"].x + incremento_x  
    if(nueva_x > 0 and nueva_x < 600):  
        personaje["rect_pos"].x = personaje["rect_pos"].x + incremento_x  
        personaje["rect"].x = personaje["rect"].x + incremento_x
```

Actualizar pantalla

Finalmente podemos crear una función llamada `actualizar_pantalla()` la cual reciba como parámetro el diccionario del personaje y la superficie de la pantalla en la cual lo dibujara

```
def actualizar_pantalla(dict_personaje, ventana_ppal):  
    ventana_ppal.blit(dict_personaje["surface"], dict_personaje["rect_pos"])
```


Nuestro modulo `homero.py` se verá de la siguiente manera

```
homero.py > ...
1  import pygame
2
3  def crear(x,y,ancho,alto):
4      dict_personaje = {}
5      dict_personaje["surface"] = pygame.image.load("imagen_homero.png")
6      dict_personaje["surface"] = pygame.transform.scale( dict_personaje["surface"], (ancho,alto))
7      dict_personaje["rect_pos"] = pygame.Rect(x,y,200,200)
8      dict_personaje["rect"] = pygame.Rect((x+ancho/2)-10,y+90,40,20)
9      dict_personaje["score"] = 0
10
11     return dict_personaje
12
13 def actualizar_pantalla(dict_personaje,ventana_ppal):
14     ventana_ppal.blit(dict_personaje["surface"],dict_personaje["rect_pos"])
15
16
17 def update(personaje,incremento_x):
18     nueva_x = personaje["rect_pos"].x + incremento_x
19     if(nueva_x > 0 and nueva_x < 600):
20         personaje["rect_pos"].x = personaje["rect_pos"].x + incremento_x
21         personaje["rect"].x = personaje["rect"].x + incremento_x
22
```

```
main_juego.py > ...
1  import pygame
2  import colores
3  import homero
4
5  ANCHO_VENTANA = 800
6  ALTO_VENTANA = 500
7
8  pygame.init()
9  ventana_ppal = pygame.display.set_mode((ANCHO_VENTANA, ALTO_VENTANA))
10 pygame.display.set_caption("PYGAME HOMERO COME DONAS")
11
12 # TIMER
13 timer = pygame.USEREVENT + 0
14 pygame.time.set_timer(timer, 100)
15
16 dict_personaje = homero.crear(ANCHO_VENTANA/2, ALTO_VENTANA-200, 200, 200)
17
18 flag_run = True
19 while flag_run:
20     lista_eventos = pygame.event.get()
21     for evento in lista_eventos:
22         if evento.type == pygame.QUIT:
23             flag_run = False
24
25     ventana_ppal.fill(colores.NEGRO)
26     # Actualizamos la pantalla del personaje
27     homero.actualizar_pantalla(dict_personaje, ventana_ppal)
28
29     pygame.display.flip()
30 pygame.quit()
```

Con estas mejoras solo nos queda importar el módulo en nuestro main y llamar a las funciones creadas, quedando de la siguiente manera

Agregar eventos de teclado

Adicionalmente podemos agregar eventos de teclado para mover y actualizar nuestra imagen usando la funcion `update()` del modulo `homero.py` para que al presionar teclas izquierda y derecha, nuestro personaje se mueva en el eje X en esas direcciones

```
flag_run = True
while flag_run:
    lista_eventos = pygame.event.get()
    for evento in lista_eventos:
        if evento.type == pygame.QUIT:
            flag_run = False

    lista_teclas = pygame.key.get_pressed()

    if lista_teclas[pygame.K_LEFT] :
        pixeles_a_mover_en_x = -10
        homero.update(dict_personaje, pixeles_a_mover_en_x)

    if lista_teclas[pygame.K_RIGHT] :
        pixeles_a_mover_en_x = 10
        homero.update(dict_personaje, pixeles_a_mover_en_x)

    ventana_ppal.fill(colores.NEGRO)
    # Actualizamos la pantalla del personaje
    homero.actualizar_pantalla(dict_personaje, ventana_ppal)

    pygame.display.flip()
pygame.quit()
```